

ΤΕΧΝΗΤΗ ΝΟΗΜΟΣΥΝΗ - ΕΡΓΑΣΙΑ 1

Βαλεντίνος Γούδας- ics23170

Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
Επιστήμη Υπολογιστών

BLOCKS WORLD

ΕΙΣΑΓΩΓΗ	3	(\n)

ΑΝΑΛΥΣΗ ΠΡΟΒΛΗΜΑΤΩΝ.	5	(\n)

ΑΝΑΖΗΤΗΣΗ ΠΡΩΤΑ ΣΕ ΠΛΑΤΟΣ.	7	(\n)

ΑΝΑΖΗΤΗΣΗ ΠΡΩΤΑ ΣΕ ΒΑΘΟΣ	11	(\n)

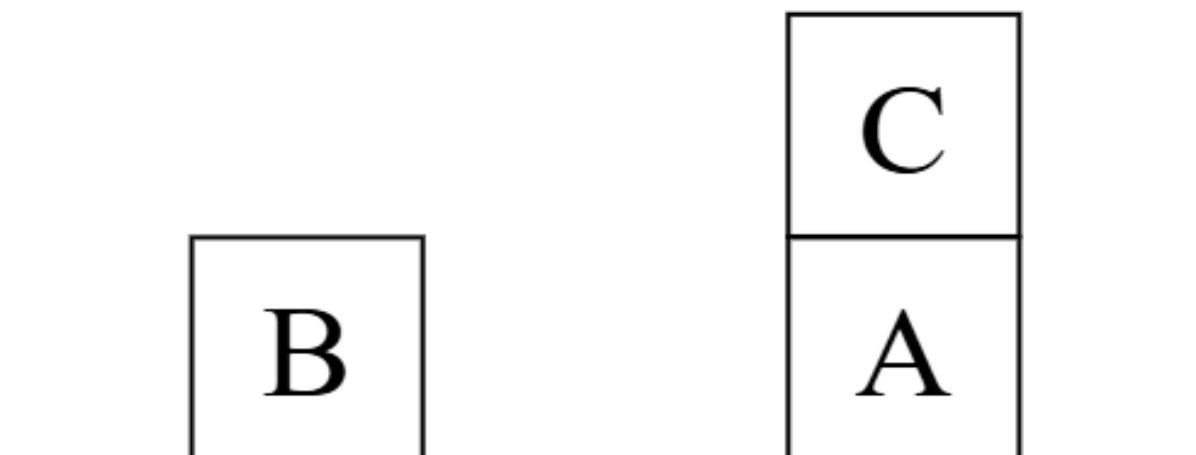
ΕΥΡΕΤΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ.	19	(\n)

ΑΝΑΖΗΤΗΣΗ ΠΡΩΤΑ ΣΤΟΝ ΚΑΛΥΤΕΡΟ.	20	(\n)

ΑΝΑΖΗΤΗΣΗ A*	25	(\n)

ΣΥΜΠΕΡΑΣΜΑΤΑ	28	(\n)

#==== ΕΙΣΑΓΩΓΗ ==== #



https://en.wikipedia.org/wiki/Blocks_world

Ο κόσμος των κύβων (Blocks World) αποτελεί ένα από τα πιο κλασικά προβλήματα στον χώρο της Τεχνητής Νοημοσύνης και ειδικότερα στον τομέα των αλγορίθμων αναζήτησης. Το πρόβλημα συνίσταται στη μετακίνηση κύβων με σκοπό την αναδιάταξή τους σε μια συγκεκριμένη λογική διάταξη, ξεκινώντας από μια αρχική κατάσταση. Οι κύβοι μπορούν είτε να τοποθετηθούν απευθείας στο τραπέζι είτε να στοιβαχθούν πάνω σε άλλους κύβους. Κάθε κύβος θεωρείται «καθαρός» όταν δεν υπάρχει άλλος κύβος τοποθετημένος πάνω του. Η μετακίνηση ενός κύβου επιτρέπεται μόνο όταν είναι καθαρός και ο προορισμός του είναι είτε το τραπέζι είτε ένας άλλος κύβος που επίσης είναι καθαρός.

Η παρούσα εργασία επικεντρώνεται στην υλοποίηση και αξιολόγηση τεσσάρων διαφορετικών αλγορίθμων αναζήτησης για την επίλυση του προβλήματος των κύβων. Συγκεκριμένα, υλοποιούνται δύο αλγόριθμοι τυφλής αναζήτησης: **Depth-First Search (DFS)** και **Breadth-First Search (BFS)**, καθώς και δύο αλγόριθμοι πληροφορικής αναζήτησης: **Best-First Search** και **A***. Οι αλγόριθμοι αυτοί καλούνται να υπολογίσουν αυτόματα την ακολουθία των ενεργειών (μετακινήσεων) που θα οδηγήσουν από την αρχική κατάσταση στην επιθυμητή τελική διάταξη, με στόχο να επιτευχθεί η λύση με το ελάχιστο δυνατό κόστος, το οποίο σε αυτήν την περίπτωση ισοδυναμεί με τον ελάχιστο αριθμό μετακινήσεων.

Για την αξιολόγηση, εξετάζεται η απόδοση κάθε αλγορίθμου σε ένα σύνολο προβλημάτων διαφορετικής δυσκολίας και μεγέθους. Πιο συγκεκριμένα, μελετάται:

- **Ικανότητα επίλυσης:** Αν ο αλγόριθμος καταφέρνει να βρει λύση για κάθε πρόβλημα μέσα στο καθορισμένο χρονικό όριο.

- **Χρόνος εκτέλεσης:** Ο συνολικός χρόνος που απαιτείται για την εύρεση της λύσης.
- **Υπολογιστική πολυπλοκότητα:** Μετρικά όπως ο αριθμός επεκτάσεων κόμβων και το μέγιστο βάθος αναζήτησης.
- **Κόστος διαδρομής:** Το συνολικό κόστος της λύσης, δηλαδή ο αριθμός των βημάτων που απαιτούνται για την επίτευξη του στόχου.

Τα αποτελέσματα των πειραμάτων για ένα δείγμα από τα προβλήματα, παρουσιάζονται συγκεντρωτικά σε πίνακα, ώστε να είναι δυνατή η άμεση σύγκριση των αλγορίθμων ως προς τα παραπάνω κριτήρια. Στη συνέχεια, ακολουθεί σχολιασμός των ευρημάτων, με στόχο να εξαχθούν γενικά συμπεράσματα για τα πλεονεκτήματα και τα μειονεκτήματα κάθε μεθόδου, καθώς και για το ποιος αλγόριθμος είναι πιο κατάλληλος σε διαφορετικές συνθήκες προβλημάτων.

Σημείωση για την εκτέλεση του κώδικα

Για την εκτέλεση του προγράμματος, ο χρήστης πρέπει να πληκτρολογήσει στο τερματικό την εντολή:

```
python      bw.py      <algorithm>      <problem>      <solution_file>
```

- Στη θέση του **algorithm** επιλέγεται ένας από τους τέσσερις διαθέσιμους αλγορίθμους: `breadth`, `depth`, `best`, ή `astar`.
- Στη θέση του **problem** δίνεται το όνομα ενός από τα δοσμένα αρχεία προβλημάτων (π.χ. `probBLOCKS-4-0.pddl`).
- Στη θέση του **solution_file** ορίζεται το όνομα του αρχείου εξόδου που θα περιέχει τη λύση, με κατάληξη `.txt`.

#==== ΑΝΑΛΥΣΗ ΠΡΟΒΛΗΜΑΤΩΝ =====#

Blocks Problems					
probBLOCKS-4-0.pddl	probBLOCKS-4-1.pddl	probBLOCKS-4-2.pddl	probBLOCKS-5-0.pddl	probBLOCKS-5-1.pddl	probBLOCKS-5-2.pddl
probBLOCKS-6-0.pddl	probBLOCKS-6-1.pddl	probBLOCKS-6-2.pddl	probBLOCKS-7-0.pddl	probBLOCKS-7-1.pddl	probBLOCKS-7-2.pddl
probBLOCKS-8-0.pddl	probBLOCKS-8-1.pddl	probBLOCKS-8-2.pddl	probBLOCKS-9-0.pddl	probBLOCKS-9-1.pddl	probBLOCKS-9-2.pddl
probBLOCKS-10-0.pddl	probBLOCKS-10-1.pddl	probBLOCKS-10-2.pddl	probBLOCKS-11-0.pddl	probBLOCKS-11-1.pddl	probBLOCKS-11-2.pddl
probBLOCKS-12-0.pddl	probBLOCKS-12-1.pddl	probBLOCKS-13-0.pddl	probBLOCKS-13-1.pddl	probBLOCKS-14-0.pddl	probBLOCKS-14-1.pddl
probBLOCKS-15-0.pddl	probBLOCKS-15-1.pddl	probBLOCKS-16-0.pddl	probBLOCKS-16-2.pddl	probBLOCKS-17-0.pddl	probBLOCKS-17-1.pddl
probBLOCKS-18-0.pddl	probBLOCKS-18-1.pddl	probBLOCKS-19-0.pddl	probBLOCKS-19-1.pddl	probBLOCKS-20-0.pddl	probBLOCKS-20-1.pddl
probBLOCKS-25-0.pddl	probBLOCKS-25-1.pddl	probBLOCKS-28-0.pddl	probBLOCKS-28-1.pddl	probBLOCKS-30-0.pddl	probBLOCKS-30-1.pddl
probBLOCKS-35-0.pddl	probBLOCKS-35-1.pddl	probBLOCKS-40-0.pddl	probBLOCKS-40-1.pddl	probBLOCKS-45-0.pddl	probBLOCKS-45-1.pddl
probBLOCKS-50-0.pddl	probBLOCKS-50-1.pddl	probBLOCKS-55-0.pddl	probBLOCKS-55-1.pddl	probBLOCKS-60-0.pddl	probBLOCKS-60-1.pddl

<http://www.cs.colostate.edu/meps/repository/aips2000.html#blocks>

Πριν εξεταστεί η λειτουργικότητα των αλγορίθμων, αξίζει να αναλυθούν τα δοσμένα προβλήματα και η μορφή τους, ώστε να εκτιμηθεί πώς η πολυπλοκότητά τους επηρεάζει την αποδοτικότητα κάθε αλγορίθμου και να εντοπιστούν οι παράγοντες που καθιστούν ένα πρόβλημα πιο δύσκολο ή πιο εύκολο.

Με λίγη μελέτη στα διάφορα προβλήματα, παρατηρείται ένα σαφές μοτίβο στην ονομασία των αρχείων:

- **Ο πρώτος αριθμός** (από 4 έως 60) δηλώνει το πλήθος των αντικειμένων στο πρόβλημα. Όσο περισσότερα αντικείμενα περιλαμβάνονται, τόσο αυξάνεται και η πολυπλοκότητα του προβλήματος (συνήθως).
- **Ο δεύτερος αριθμός** (0, 1 ή 2) υποδηλώνει τη μορφή της αρχικής κατάστασης:
 - Στα προβλήματα με **δεύτερο αριθμό 0**, όλα τα αντικείμενα ξεκινούν τοποθετημένα πάνω στο τραπέζι.
 - Στα προβλήματα με **δεύτερο αριθμό 1**, όλα τα αντικείμενα βρίσκονται σε μία στοίβα (π.χ. B πάνω σε C, C πάνω σε A, A πάνω σε D), με μόνο το ανώτερο αντικείμενο να είναι CLEAR.
 - Στα προβλήματα με **δεύτερο αριθμό 2**, η αρχική διάταξη περιλαμβάνει μερικές στοίβες: κάποια αντικείμενα βρίσκονται στο τραπέζι, ενώ άλλα είναι τοποθετημένα πάνω σε άλλα αντικείμενα.

(Είναι πιθανό ο δεύτερος αριθμός να σχετίζεται και με το πλήθος των αντικειμένων που βρίσκονται σε λανθασμένη θέση ή με την «απόσταση» από τον στόχο, αν και κάτι τέτοιο δεν μπόρεσε να επιβεβαιωθεί πλήρως.)

Όσον αφορά τη δυσκολία, μπορεί να υποτεθεί (γενική εκτίμηση, όχι απόλυτο για κάθε πρόβλημα) ότι:

- Τα προβλήματα τύπου 0 (όλα στο τραπέζι) είναι χαμηλής δυσκολίας.
- Τα προβλήματα τύπου 1 (ενιαία στοίβα) είναι υψηλής δυσκολίας.
- Τα προβλήματα τύπου 2 (μερικές στοίβες) παρουσιάζουν ενδιάμεση δυσκολία.

Παράδειγματα

Το αρχείο probBLOCKS-4-0 είναι ένα πρόβλημα με 4 αντικείμενα που ξεκινάν όλα στο τραπέζι

```
(define (problem BLOCKS-4-0)
(:domain BLOCKS)
(:objects D B A C )
(:INIT (CLEAR C) (CLEAR A) (CLEAR B) (CLEAR D) (ONTABLE C) (ONTABLE A)
(ONTABLE B) (ONTABLE D) (HANDEEMPTY))
(:goal (AND (ON D C) (ON C B) (ON B A)))
)
```

Το αρχείο probBLOCKS-4-1 είναι ένα πρόβλημα με 4 αντικείμενα που διαμορφώνουν μία στοίβα

```
(define (problem BLOCKS-4-1)
(:domain BLOCKS)
(:objects A C D B )
(:INIT (CLEAR B) (ONTABLE D) (ON B C) (ON C A) (ON A D) (HANDEEMPTY))
(:goal (AND (ON D C) (ON C A) (ON A B)))
)
```

Το αρχείο probBLOCKS-4-2 είναι ένα πρόβλημα με 4 αντικείμενα και υπάρχει μία στοίβα μεταξύ δύο αντικειμένων

```
(define (problem BLOCKS-4-2)
(:domain BLOCKS)
(:objects B D C A )
(:INIT (CLEAR A) (CLEAR C) (CLEAR D) (ONTABLE A) (ONTABLE B) (ONTABLE D)
(ON C B) (HANDEEMPTY))
(:goal (AND (ON A B) (ON B C) (ON C D)))
)
```

#==== ΑΝΑΖΗΤΗΣΗ ΠΡΩΤΑ ΣΕ ΠΛΑΤΟΣ ====

Ο αλγόριθμος BFS (Breadth-First Search), όπως ήταν αναμενόμενο, δεν καταφέρνει να επιλύσει το μεγαλύτερο σύνολο προβλημάτων μέσα στο προκαθορισμένο χρονικό όριο. Στη συνέχεια παρουσιάζεται η ανάλυση της απόδοσής του, με βάση τον τύπο κάθε προβλήματος.

Αρχική κατάσταση τύπου 0 (όλα τα αντικείμενα στο τραπέζι)

Στην κατηγορία χαμηλής δυσκολίας, ο αλγόριθμος αναζήτησης κατά πλάτος καταφέρνει να επιλύσει προβλήματα έως και το probBLOCKS-8-0, δηλαδή περιπτώσεις που περιλαμβάνουν έως 8 αντικείμενα. Ακολουθούνε screenshots της συμπεριφοράς του αλγορίθμου μέχρι εκείνο το πρόβλημα:

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-4-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 3
Κόμβοι που επεκτάθηκαν: 58
Μέγιστο βάθος αναζήτησης: 3
Χρόνος εκτέλεσης: 0.0012881755828857422
Η λύση είναι έγκυρη: NAI

C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-5-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 6
Κόμβοι που επεκτάθηκαν: 422
Μέγιστο βάθος αναζήτησης: 6
Χρόνος εκτέλεσης: 0.006810903549194336
Η λύση είναι έγκυρη: NAI

C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-6-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 6
Κόμβοι που επεκτάθηκαν: 2126
Μέγιστο βάθος αναζήτησης: 6
Χρόνος εκτέλεσης: 0.055489301681518555
Η λύση είναι έγκυρη: NAI

C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-7-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 10
Κόμβοι που επεκτάθηκαν: 26918
Μέγιστο βάθος αναζήτησης: 10
Χρόνος εκτέλεσης: 1.1273305416107178
Η λύση είναι έγκυρη: NAI

C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-8-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 9
Κόμβοι που επεκτάθηκαν: 340131
Μέγιστο βάθος αναζήτησης: 9
Χρόνος εκτέλεσης: 19.314778089523315
Η λύση είναι έγκυρη: NAI

C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-9-0.pddl sol.txt
FAILURE
```

Τα αποτελέσματα επιβεβαιώνουν τον τυπικό χαρακτήρα του αλγορίθμου BFS. Η στρατηγική του να εξερευνά εξαντλητικά όλα τα επίπεδα πριν προχωρήσει στο επόμενο

εξασφαλίζει βέλτιστη λύση, αλλά έχει σημαντικό τίμημα: το κόστος σε μνήμη και χρόνο αυξάνεται εκθετικά όσο μεγαλώνει η δυσκολία του προβλήματος. Αυτό γίνεται ιδιαίτερα εμφανές συγκρίνοντας το **probBLOCKS-7-0**, όπου επεκτάθηκαν μόλις **26.918 κόμβοι** και χρειάστηκε **1s**, με το **probBLOCKS-8-0**, που απαιτούσε **340.131 κόμβους** και σχεδόν **19s**, για ένα αντικείμενο περισσότερο. Ακόμη πιο χαρακτηριστικό είναι ότι το **probBLOCKS-9-0** δεν ολοκληρώθηκε καν εντός του χρονικού ορίου των 60s, γεγονός που υπογραμμίζει πόσο δραματικά αυξάνεται η πολυπλοκότητα ακόμη και με μία μικρή αύξηση στο μέγεθος του προβλήματος.

Με άλλα λόγια, επιβεβαιώνεται η θεωρητική σχέση της χρονικής και χωρικής πολυπλοκότητας του BFS, η οποία είναι $O(b^d)$, όπου:

- **b**: ο μέσος παράγοντας διακλάδωσης (δηλαδή ο αριθμός διαδοχών ανά κόμβο)
- **d**: το βάθος της λύσης στο δέντρο αναζήτησης.

Αν εξετάσουμε τα παρακάτω παραδείγματα (4-0, 6-0, 8-0), η τάση γίνεται ξεκάθαρη:

- **4 αντικείμενα** → βάθος = **3**, κόμβοι που επεκτάθηκαν = **58**, χρόνος $\approx$ **0,001s**
- **6 αντικείμενα** → βάθος = **6**, κόμβοι που επεκτάθηκαν = **2126**, χρόνος $\approx$ **0,05s**
- **8 αντικείμενα** → βάθος = **9**, κόμβοι που επεκτάθηκαν = **340131**, χρόνος $\approx$ **19s**

Οι κόμβοι που επεκτάθηκαν συνδέονται άμεσα με τη χωρική πολυπλοκότητα, καθώς ουσιαστικά όλοι αυτοί οι κόμβοι παραμένουν στη μνήμη μέχρι να ολοκληρωθεί η επίλυση. Στα παραπάνω παραδείγματα, ο αριθμός τους αυξανόταν δραματικά: $58 \rightarrow 2126 \rightarrow 340.131$ για βάθη $3 \rightarrow 6 \rightarrow 9$. Μια γρήγορη εκτίμηση δείχνει ότι ο παράγοντας διακλάδωσης b κυμαίνεται περίπου μεταξύ 3 και 4, και αν θεωρήσουμε ότι παραμένει σταθερός, τότε η αύξηση του βάθους d είναι αυτή που εκτοξεύει εκθετικά τη συνολική πολυπλοκότητα.

Συνεπώς, ακόμη και μια σχετικά μικρή αύξηση στον αριθμό των αντικειμένων, που ενδεχομένως οδηγεί σε μεγαλύτερο βάθος δέντρου, προκαλεί δραματική αύξηση τόσο στο χρόνο όσο και στη μνήμη που απαιτεί ο αλγόριθμος. Το φαινόμενο αυτό παρατηρείται ανεξάρτητα από τον τύπο του προβλήματος, είτε πρόκειται για τύπο 0, 1 ή 2.

Αρχική κατάσταση τύπου 1 (ενιαία στοίβα)

Στην κατηγορία υψηλής δυσκολίας, ο αλγόριθμος BFS εμφανίζει ανάλογη συμπεριφορά. Καταφέρνει να επιλύσει προβλήματα έως το probBLOCKS-8-1 (8 αντικείμενα), με τα αποτελέσματα να παρουσιάζουν μόνο μικρές αποκλίσεις σε σχέση με τα προβλήματα χαμηλής δυσκολίας. Αυτό μπορεί να αποδοθεί στη στρατηγική εξαντλητικής εξερεύνησης

κάθε επιπέδου, ενώ μπορεί κιόλας να υποδηλώνει ότι οι συγκεκριμένες αρχικές καταστάσεις δεν διέφεραν σημαντικά στη συνολική δυσκολία (εξαρτάται και από τον στόχο). Ακολουθούνε screenshots της συμπεριφοράς του αλγορίθμου μέχρι εκείνο το πρόβλημα:

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-4-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 5
Κόμβοι που επεκτάθηκαν: 54
Μέγιστο βάθος αναζήτησης: 5
Χρόνος εκτέλεσης: 0.0012230873107910156
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-5-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 5
Κόμβοι που επεκτάθηκαν: 412
Μέγιστο βάθος αναζήτησης: 5
Χρόνος εκτέλεσης: 0.006196498870849609
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-6-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 5
Κόμβοι που επεκτάθηκαν: 3519
Μέγιστο βάθος αναζήτησης: 5
Χρόνος εκτέλεσης: 0.06923604011535645
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-7-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 11
Κόμβοι που επεκτάθηκαν: 36933
Μέγιστο βάθος αναζήτησης: 11
Χρόνος εκτέλεσης: 1.2349755764007568
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-8-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 10
Κόμβοι που επεκτάθηκαν: 382060
Μέγιστο βάθος αναζήτησης: 10
Χρόνος εκτέλεσης: 18.574490308761597
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-9-1.pddl sol.txt
FAILURE
```

Ο BFS δεν φαίνεται να επηρεάζεται ουσιαστικά από την εκτιμημένη δυσκολία των διαφορετικών τύπων προβλημάτων. Όσα παρατηρήθηκαν προηγουμένως όσον αφορά την εκθετική αύξηση της πολυπλοκότητας και τη σχέση με τον αριθμό των αντικειμένων ισχύουν και εδώ.

Αρχική κατάσταση τύπου 2 (μερικές στοίβες)

Αντίστοιχα και εδώ, τα αποτελέσματα είναι παρόμοια. Ο αλγόριθμος λύνει και πάλι μέχρι το πρόβλημα των 8 αντικειμένων probBLOCKS-8-2, με τιμές περίπου ίδιες με τις προηγούμενες δυσκολίες.

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-4-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 3  
Κόμβοι που επεκτάθηκαν: 52  
Μέγιστο βάθος αναζήτησης: 3  
Χρόνος εκτέλεσης: 0.0012791156768798828  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-5-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 8  
Κόμβοι που επεκτάθηκαν: 447  
Μέγιστο βάθος αναζήτησης: 8  
Χρόνος εκτέλεσης: 0.006738424301147461  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-6-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 10  
Κόμβοι που επεκτάθηκαν: 3946  
Μέγιστο βάθος αναζήτησης: 10  
Χρόνος εκτέλεσης: 0.06867194175720215  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-7-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 10  
Κόμβοι που επεκτάθηκαν: 35462  
Μέγιστο βάθος αναζήτησης: 10  
Χρόνος εκτέλεσης: 1.2143704891204834  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-8-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 8  
Κόμβοι που επεκτάθηκαν: 314499  
Μέγιστο βάθος αναζήτησης: 8  
Χρόνος εκτέλεσης: 17.534644842147827  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-9-2.pddl sol.txt  
FAILURE
```

Ο ΑΛΓΟΡΙΘΜΟΣ ΑΝΑΖΗΤΗΣΗΣ ΠΡΩΤΑ ΣΕ ΠΛΑΤΟΣ ΜΠΟΡΕΙ ΝΑ ΕΠΙΛΥΣΕΙ ΟΛΑ ΤΑ ΠΡΟΒΛΗΜΑΤΑ ΜΕ ΠΛΗΘΟΣ 8 ΑΝΤΙΚΕΙΜΕΝΩΝ ΚΑΙ ΚΑΤΩ, ΚΑΙ ΣΕ ΚΑΘΕ ΤΥΠΟΥ ΠΡΟΒΛΗΜΑΤΟΣ ΤΑ ΑΠΟΤΕΛΕΣΜΑΤΑ ΕΙΝΑΙ ΠΑΡΟΜΟΙΑ. ΒΡΙΣΚΕΙ ΤΗΝ ΒΕΛΤΙΣΤΗ ΛΥΣΗ,

ΩΣΤΟΣΟ ΧΡΕΙΑΖΕΤΑΙ ΣΗΜΑΝΤΙΚΑ ΠΕΡΙΣΣΟΤΕΡΗ ΜΝΗΜΗ ΚΑΙ ΧΡΟΝΟ ΟΣΟ ΑΥΞΑΝΟΝΤΑΙ ΤΑ ΑΝΤΙΚΕΙΜΕΝΑ #

#==== ΑΝΑΖΗΤΗΣΗ ΠΡΩΤΑ ΣΕ ΒΑΘΟΣ ====#

Ο αλγόριθμος DFS (Αναζήτηση Πρώτα σε Βάθος) μπορεί να επιλύσει τα ίδια προβλήματα με τον BFS, αλλά οι τιμές των αποτελεσμάτων παρουσιάζουν σημαντικές διαφορές. Παρακάτω παρατίθεται αναλυτική παρουσίαση της απόδοσης του αλγορίθμου, με βάση τον τύπο κάθε προβλήματος.

Αρχική κατάσταση τύπου 0 (όλα τα αντικείμενα στο τραπέζι)

Στην κατηγορία χαμηλής δυσκολίας, ο αλγόριθμος αναζήτησης κατά βάθος καταφέρνει να επιλύσει προβλήματα έως και το probBLOCKS-8-0, δηλαδή περιπτώσεις που περιλαμβάνουν έως 8 αντικείμενα, όπως ο πρώτα σε πλάτος. Ακολουθούν screenshots της συμπεριφοράς του αλγορίθμου μέχρι εκείνο το πρόβλημα:

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-4-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 3
Κόμβοι που επεκτάθηκαν: 50
Μέγιστο βάθος αναζήτησης: 4
Χρόνος εκτέλεσης: 0.0007185935974121094
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-5-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 32
Κόμβοι που επεκτάθηκαν: 47
Μέγιστο βάθος αναζήτησης: 32
Χρόνος εκτέλεσης: 0.0010004043579101562
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-6-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 37
Κόμβοι που επεκτάθηκαν: 43
Μέγιστο βάθος αναζήτησης: 37
Χρόνος εκτέλεσης: 0.0015070438385009766
Η λύση είναι έγκυρη: NAI
```

```

C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-7-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 1962
Κόμβοι που επεκτάθηκαν: 29514
Μέγιστο βάθος αναζήτησης: 6814
Χρόνος εκτέλεσης: 0.8259487152099609
Η λύση είναι έγκυρη: NAI

C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-8-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 48198
Κόμβοι που επεκτάθηκαν: 221825
Μέγιστο βάθος αναζήτησης: 74389
Χρόνος εκτέλεσης: 8.624267339706421
Η λύση είναι έγκυρη: NAI

C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-9-0.pddl sol.txt
FAILURE

```

Υπάρχουν αρκετές διαφορές σε σχέση με τον BFS, οι οποίες οφείλονται κυρίως στη διαφορετική στρατηγική του DFS. Ένα από τα πιο χαρακτηριστικά σημεία που παρατηρούνται σε σχέση με πριν, είναι ότι το κόστος διαδρομής (δηλαδή το βάθος της λύσης) δεν συμπίπτει με το μέγιστο βάθος αναζήτησης, σε αντίθεση με τον BFS όπου οι δύο τιμές ήταν ίδιες.

Αυτό εξηγείται από τον τρόπο που λειτουργεί ο DFS: αντί να εξερευνά πλήρως κάθε επίπεδο πριν προχωρήσει στο επόμενο, βυθίζεται όσο το δυνατόν βαθύτερα σε έναν κλάδο πριν επιστρέψει για να εξετάσει άλλους.

Για παράδειγμα, αν η λύση βρίσκεται σε βάθος 2, ο DFS μπορεί να ακολουθήσει έναν κλάδο μέχρι το βάθος 3 (ή και περισσότερο) πριν επιστρέψει, και στη συνέχεια να εντοπίσει τη λύση σε κάποιον άλλο κλάδο στο σωστό βάθος. Στην περίπτωση αυτή, το κόστος της διαδρομής θα είναι 2 ενώ το μέγιστο βάθος αναζήτησης θα είναι 3.

Αυτή η συμπεριφορά του DFS έχει τόσο πλεονεκτήματα όσο και μειονεκτήματα.

Από τη μία πλευρά, **δεν εγγυάται βέλτιστη λύση**: στο προηγούμενο παράδειγμα, αν εντόπιζε λύση σε βάθος 3, θα την αποδεχόταν, ακόμα κι αν υπήρχε καλύτερη λύση σε βάθος 2.

Από την άλλη πλευρά, το μεγάλο πλεονέκτημα του DFS έναντι του BFS είναι η **πολύ χαμηλότερη χωρική πολυπλοκότητα**. Ο DFS χρειάζεται να αποθηκεύει μόνο τους κόμβους της τρέχουσας διαδρομής και τους αδελφούς κόμβους που δεν έχουν ακόμα εξερευνηθεί, με αποτέλεσμα η χρήση μνήμης να είναι δραματικά μικρότερη σε σχέση με τον BFS, που πρέπει να κρατά στη μνήμη ολόκληρα επίπεδα του δέντρου.

Ωστόσο, σε αυτήν την περίπτωση, ο αριθμός των κόμβων που επεκτάθηκαν δεν αντικατοπτρίζει την πραγματική μνήμη που απαιτήθηκε, καθώς οι κόμβοι που υφίστανται `expand()` θεωρητικά δεν παραμένουν όλοι ταυτόχρονα στην μνήμη - πολλοί διαγράφονται. Η χωρική πολυπλοκότητα του DFS είναι $O(b \cdot m)$, όπου m το βάθος της λύσης και b ο παράγοντας διακλάδωσης.

Προηγουμένως είχε γίνει μια εκτίμηση για τον παράγοντα διακλάδωσης b των προβλημάτων, θεωρώντας ότι κυμαίνεται περίπου μεταξύ 3 και 4. Παρακάτω παρουσιάζεται η αντίστοιχη χωρική πολυπλοκότητα για κάθε πρόβλημα, υπολογισμένη τόσο για $b=3$ όσο και για $b=4$.

ProbBlocks-4-0:

- $b=3, m=4, b \cdot m = 12$ (< Κόμβοι που επεκτάθηκαν BFS = 58)
- $b=4, m=4, b \cdot m = 16$ (< Κόμβοι που επεκτάθηκαν BFS = 58)

ProbBlocks-5-0:

- $b=3, m=32, b \cdot m = 96$ (< Κόμβοι που επεκτάθηκαν BFS = 422)
- $b=4, m=32, b \cdot m = 128$ (< Κόμβοι που επεκτάθηκαν BFS = 422)

ProbBlocks-6-0:

- $b=3, m=37, b \cdot m = 111$ (< Κόμβοι που επεκτάθηκαν BFS = 2126)
- $b=4, m=37, b \cdot m = 148$ (< Κόμβοι που επεκτάθηκαν BFS = 2126)

ProbBlocks-7-0:

- $b=3, m=6814, b \cdot m = 20.442$ (< Κόμβοι που επεκτάθηκαν BFS = 26.918)
- $b=4, m=6814, b \cdot m = 27.256$ (> Κόμβοι που επεκτάθηκαν BFS = 26.918)

ProbBlocks-8-0:

- $b=3, m=74.389, b \cdot m = 223.167$ (< Κόμβοι που επεκτάθηκαν BFS = 340.131)
- $b=4, m=74.389, b \cdot m = 297.556$ (< Κόμβοι που επεκτάθηκαν BFS = 340.131)

Γίνεται σαφές η σημαντικά μειωμένη χωρική πολυπλοκότητα του DFS σε σύγκριση με τον BFS.

Αν και γενικά δεν θεωρείται πως ο DFS υπερσχύει και σε χρονική πολυπλοκότητα, στα παραπάνω προβλήματα φαίνεται να κάνει καλύτερους χρόνους. Η πολυπλοκότητα χρόνου του DFS είναι $O(b^m)$ όπου m συνήθως είναι μεγαλύτερο από το d (βάθος λύσης). Ωστόσο και οι δύο αλγόριθμοι έχουν χρονική πολυπλοκότητα εκθετικής τάξης

Οι καλύτεροι χρόνοι του DFS μπορεί να οφείλονται είτε στη φύση των ίδιων των προβλημάτων (δηλαδή τυχαία βρήκε τη λύση νωρίτερα) είτε στη χαμηλότερη κατανάλωση μνήμης του αλγορίθμου σε σχέση με τον BFS. Η επίδραση αυτή ωστόσο εξαρτάται από το πώς οι timers της Python μετρούν τον χρόνο εκτέλεσης και αν περιλαμβάνουν καθυστερήσεις που προκαλούνται από υψηλό φόρτο μνήμης.

Αρχική κατάσταση τύπου 1 (ενιαία στοίβα)

Στην κατηγορία υψηλής δυσκολίας, ο αλγόριθμος DFS καταφέρνει και πάλι να επιλύσει προβλήματα έως και το probBLOCKS-8-1 (8 αντικείμενα). Ωστόσο, οι τιμές των αποτελεσμάτων παρουσιάζουν σημαντικές διαφοροποιήσεις σε σχέση με τον τύπο 0, γεγονός που υποδηλώνει ότι ο τύπος του προβλήματος μπορεί να επηρεάζει την αποδοτικότητα του αλγορίθμου. Ακολουθούν screenshots που απεικονίζουν τη συμπεριφορά του DFS έως και το συγκεκριμένο πρόβλημα.

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-4-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 7
Κόμβοι που επεκτάθηκαν: 20
Μέγιστο βάθος αναζήτησης: 7
Χρόνος εκτέλεσης: 0.0003376007080078125
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-5-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 76
Κόμβοι που επεκτάθηκαν: 179
Μέγιστο βάθος αναζήτησης: 76
Χρόνος εκτέλεσης: 0.002603769302368164
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-6-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 570
Κόμβοι που επεκτάθηκαν: 1872
Μέγιστο βάθος αναζήτησης: 625
Χρόνος εκτέλεσης: 0.029834747314453125
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-7-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 3260
Κόμβοι που επεκτάθηκαν: 4478
Μέγιστο βάθος αναζήτησης: 3260
Χρόνος εκτέλεσης: 0.13446807861328125
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-8-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 72406
Κόμβοι που επεκτάθηκαν: 146571
Μέγιστο βάθος αναζήτησης: 74449
Χρόνος εκτέλεσης: 6.233591556549072
Η λύση είναι έγκυρη: ΝΑΙ

C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-9-1.pddl sol.txt
FAILURE
```

Οι διαφορές επηρεάζουν τόσο τη μνήμη (μέγιστο βάθος) όσο και τον χρόνο εκτέλεσης, με τις πιο εμφανείς διαφοροποιήσεις να παρατηρούνται στο μέγιστο βάθος. Συγκρίνοντας τον τύπο 0 με τον τύπο 1, έχουμε:

- Μέγιστο βάθος (4-0) = 4 < Μέγιστο βάθος (4-1) = 7
- Μέγιστο βάθος (5-0) = 32 < Μέγιστο βάθος (5-1) = 76
- Μέγιστο βάθος (6-0) = 37 < Μέγιστο βάθος (6-1) = 625
- Μέγιστο βάθος (7-0) = 6814 > Μέγιστο βάθος (7-1) = 3260
- Μέγιστο βάθος (8-0) = 74.389 < Μέγιστο βάθος (8-1) = 74.449

Σε όλα τα προβλήματα εκτός αυτών με τα 7 αντικείμενα, ο αλγόριθμος έχει μεγαλύτερο μέγιστο βάθος, δηλαδή μεγαλύτερη πολυπλοκότητα μνήμης. Αυτό είναι λογικό για την συμπεριφορά του αλγορίθμου, αφού στα προβλήματα τύπου 0 (όλα στο τραπέζι) οι λύσεις τυπικά βρίσκονται σε σχετικά μικρό βάθος, οπότε ο DFS μπορεί να βρει τη λύση εξερευνώντας έναν μικρό αριθμό κλάδων, και το μέγιστο βάθος παραμένει σχετικά χαμηλό.

Αντίθετα, στα προβλήματα τύπου 1 (όλα σε στοίβα), η λύση μπορεί να απαιτεί να "ξεστοιβάξει" πολλούς κύβους πριν επανατοποθετήσει τα σωστά αντικείμενα, γεγονός που αναγκάζει τον DFS να φτάσει σε πολύ μεγαλύτερο βάθος σε κάποιους κλάδους πριν βρει τη λύση.

Επομένως, συμπεραίνουμε πως ο τύπος προβλήματος/αρχικής κατάστασης μπορεί να επηρεάσει τον DFS, αν και πολλές φορές εξαρτάται και από το ίδιο το πρόβλημα.

Αρχική κατάσταση τύπου 2 (μερικές στοίβες)

Ομοίως, και για την κατηγορία προβλημάτων τύπου 2, τα αποτελέσματα παρουσιάζουν διαφοροποιήσεις σε σχέση με τους προηγούμενους τύπους. Ο αλγόριθμος εξακολουθεί να επιλύει τα προβλήματα έως και το probBLOCKS-8-2 (8 αντικείμενα), και ότι

συμπεράσματα έχουν εξαχθεί ως τώρα ισχύουν και εδώ. Ακολουθούνε screenshots της συμπεριφοράς του αλγορίθμου μέχρι εκείνο το πρόβλημα:

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-4-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 8  
Κόμβοι που επεκτάθηκαν: 52  
Μέγιστο βάθος αναζήτησης: 11  
Χρόνος εκτέλεσης: 0.0009033679962158203  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-5-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 68  
Κόμβοι που επεκτάθηκαν: 116  
Μέγιστο βάθος αναζήτησης: 68  
Χρόνος εκτέλεσης: 0.0019218921661376953  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-6-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 61  
Κόμβοι που επεκτάθηκαν: 3760  
Μέγιστο βάθος αναζήτησης: 638  
Χρόνος εκτέλεσης: 0.06470847129821777  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-7-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 5313  
Κόμβοι που επεκτάθηκαν: 19305  
Μέγιστο βάθος αναζήτησης: 6645  
Χρόνος εκτέλεσης: 0.4967031478881836  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-8-2.pddl sol.txt  
SUCCESS
```

```
Κόστος διαδρομής: 58349  
Κόμβοι που επεκτάθηκαν: 193843  
Μέγιστο βάθος αναζήτησης: 74574  
Χρόνος εκτέλεσης: 8.04034423828125  
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py depth probBLOCKS-9-2.pddl sol.txt  
FAILURE
```

Ο ΑΛΓΟΡΙΘΜΟΣ ΑΝΑΖΗΤΗΣΗΣ ΠΡΩΤΑ ΣΕ ΒΑΘΟΣ ΜΠΟΡΕΙ ΝΑ ΕΠΙΛΥΣΕΙ ΟΛΑ ΤΑ ΠΡΟΒΛΗΜΑΤΑ ΜΕ ΕΩΣ 8 ΑΝΤΙΚΕΙΜΕΝΑ, ΑΛΛΑ ΕΜΦΑΝΙΖΕΙ ΜΕΓΑΛΥΤΕΡΗ ΕΥΑΙΣΘΗΣΙΑ ΣΤΗ ΔΥΣΚΟΛΙΑ ΑΝΑΛΟΓΑ ΜΕ ΤΟΝ ΤΥΠΟ ΤΟΥ ΠΡΟΒΛΗΜΑΤΟΣ. ΑΝ ΚΑΙ ΔΕΝ ΕΓΓΥΑΤΑΙ ΤΗΝ ΕΥΡΕΣΗ ΤΗΣ ΒΕΛΤΙΣΤΗΣ ΛΥΣΗΣ, ΠΡΟΣΦΕΡΕΙ ΣΗΜΑΝΤΙΚΟ ΠΛΕΟΝΕΚΤΗΜΑ ΟΣΟΝ ΑΦΟΡΑ ΤΗΝ ΚΑΤΑΝΑΛΩΣΗ ΜΝΗΜΗΣ.

#==== ΕΥΡΕΤΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ ====#

Στον κώδικα υλοποιήθηκαν τρεις ευρετικές συναρτήσεις:

Heuristic 1 (h1)

Η H1 υπολογίζει τον αριθμό των blocks που δεν βρίσκονται στη σωστή θέση, συγκρίνοντας την τρέχουσα διάταξη με την τελική. Συγκεκριμένα, για κάθε block ελέγχεται αν βρίσκεται πάνω στο σωστό block (ή στο τραπέζι) όπως στο goal state. Αν όχι, αυξάνεται το κόστος κατά 1. Χρησιμοποιείται για να ξεκινήσει με μια απλή εκτίμηση πριν εφαρμοστούν οι πιο περίπλοκες ευρετικές υπολογίζοντας το $f(n)$ στην αρχή τόσο για τον A* όσο και για τον Best-First Search

Heuristic 2 (h2)

Η H2 εκτιμά το κόστος βασιζόμενη στο πόσες φορές πρέπει να μετακινηθεί κάθε block:

- 2 μονάδες για κάθε block που πρέπει να μετακινηθεί μία φορά.
- 4 μονάδες για κάθε block που πρέπει να μετακινηθεί δύο φορές (π.χ. επειδή βρίσκεται πάνω σε λάθος block που επίσης πρέπει να μετακινηθεί).

Έτσι, δίνει μεγαλύτερο βάρος σε πολύπλοκες κινήσεις όπου απαιτούνται επιπλέον ενέργειες για να ελευθερωθούν block. Χρησιμοποιείται από τον Best First και αυτό γιατί παρέχει πιο ακριβή πρόβλεψη του κόστους μέχρι τον στόχο, αφού ενσωματώνει τη δυσκολία του ξεμπλοκαρίσματος και όχι μόνο την απλή διαφορά θέσης.

Heuristic 3 (h3)

Η H3 είναι μια καλύτερη εκδοχή της H1. Υπολογίζει τις διαφορές μεταξύ της τρέχουσας και της τελικής θέσης λαμβάνοντας υπόψη και τα δύο επίπεδα σχέσης: ποιο block είναι από πάνω και ποιο από κάτω.

- Αν το block δεν είναι ούτε πάνω στο σωστό block ούτε κάτω από το σωστό block, προσθέτει 2 μονάδες.
- Αν είναι λάθος μόνο σε ένα από τα δύο, προσθέτει 1 μονάδα.

Χρησιμοποιείται από τον A^* , επειδή αρχικά είναι παραδεκτή (δεν υπερεκτιμά σε σχέση με το πραγματικό κόστος, όπως θα ίσχυε αν χρησιμοποιούσε την h_2) οπότε παραμένει η εγγύηση της βέλτιστης λύσης από τον A^* , και επειδή ταυτόχρονα δεν είναι τόσο απλή (όπως η h_1).

#==== ΑΝΑΖΗΤΗΣΗ ΠΡΩΤΑ ΣΤΟΝ ΚΑΛΥΤΕΡΟ =====#

Ως αλγόριθμος πληροφορημένης αναζήτησης, ο πρώτα στον καλύτερο λύνει το μεγαλύτερο εύρος των προβλημάτων. Για την επίδειξη της λειτουργίας του θα χρησιμοποιηθεί ένα δείγμα από προβλήματα με ουσιαστικό (όχι μικρό) πλήθος αντικειμένων.

Αρχική κατάσταση τύπου 0 (όλα τα αντικείμενα στο τραπέζι)

Ο αλγόριθμος σταματά να βρίσκει λύσεις για προβλήματα από το probBLOCKS-35-0 και μετά, δηλαδή για περιπτώσεις με 35 αντικείμενα και άνω. Οι τιμές των αποτελεσμάτων από εδώ και στο εξής θα εξαρτώνται κυρίως από το πόσο αποτελεσματική είναι η εκτίμηση της ευρετικής συνάρτησης. Παρακάτω παρατίθενται ενδεικτικά παραδείγματα εκτέλεσης:

```
C:\mathimata\BlocksWorld>python bw.py best probBLOCKS-20-0.pddl sol.txt
SUCCESS
```

```
Κόστος διαδρομής: 36
Κόμβοι που επεκτάθηκαν: 213
Μέγιστο βάθος αναζήτησης: 36
Χρόνος εκτέλεσης: 1.316812515258789
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py best probBLOCKS-25-0.pddl sol.txt
SUCCESS
```

```
Κόστος διαδρομής: 45
Κόμβοι που επεκτάθηκαν: 298
Μέγιστο βάθος αναζήτησης: 45
Χρόνος εκτέλεσης: 1.8627326488494873
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py best probBLOCKS-28-0.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 51

Κόμβοι που επεκτάθηκαν: 401

Μέγιστο βάθος αναζήτησης: 51

Χρόνος εκτέλεσης: 3.5478923320770264

Η λύση είναι έγκυρη: NAI

```
C:\mathimata\BlocksWorld>python bw.py best probBLOCKS-30-0.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 52

Κόμβοι που επεκτάθηκαν: 448

Μέγιστο βάθος αναζήτησης: 52

Χρόνος εκτέλεσης: 7.7347681522369385

Η λύση είναι έγκυρη: NAI

Αρχικά, η ευρετική συνάρτηση h_2 σε αυτόν τον τύπο προβλημάτων δεν διαφέρει ουσιαστικά από την h_1 (είναι σαν την h_1 με συντελεστή) γιατί πολύ απλά δεν υπάρχουν blocks που πρέπει να μετακινηθούν δύο φορές, κανένα block δεν ξεκινάει πάνω σε block που πρέπει να αλλάξει θέση, όλα ξεκινάνε στο τραπέζι. Με άλλα λόγια, η διάκριση μεταξύ «μία φορά» και «δύο φορές» δεν αξιοποιείται, γιατί δεν υπάρχουν περιπτώσεις που απαιτούν δεύτερη μετακίνηση.

Ωστόσο, αυτό δεν φάνηκε να αποτρέπει σημαντικά τον αλγόριθμο, γιατί στα παραπάνω αποτελέσματα η αύξηση του κόστους και του χρόνου σε σχέση με τα άλματα στην αύξηση των αντικειμένων, είναι γραμμική. Αυτό σηματοδοτεί ότι η ευρετική έκανε “καλές” προβλέψεις και δεν εκτόξευσε εκθετικά το κόστος.

Με άλλα λόγια, χάρη σε καλή επιλογή ευρετικής συνάρτησης, αποφεύχθηκε η χειρότερη περίπτωση χρονικής και χωρικής πολυπλοκότητας $O(b^d)$.

Πριν προχωρήσουμε στην ανάλυση των υπόλοιπων τύπων προβλημάτων, αξίζει να παρουσιαστεί και το ακόλουθο παράδειγμα εκτέλεσης:

```
C:\mathimata\BlocksWorld> python bw.py best probBLOCKS-8-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 10
Κόμβοι που επεκτάθηκαν: 33
Μέγιστο βάθος αναζήτησης: 10
Χρόνος εκτέλεσης: 0.006841897964477539
Η λύση είναι έγκυρη: NAI

C:\mathimata\BlocksWorld>python bw.py breadth probBLOCKS-8-0.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 9
Κόμβοι που επεκτάθηκαν: 340131
Μέγιστο βάθος αναζήτησης: 9
Χρόνος εκτέλεσης: 17.780848264694214
Η λύση είναι έγκυρη: NAI
```

Όπως έχει επισημανθεί, ο αλγόριθμος BFS είναι βέλτιστος, δηλαδή εξασφαλίζει την εύρεση της βέλτιστης λύσης. Συγκρίνοντας τον αλγόριθμο Best-First με τον BFS στο ίδιο πρόβλημα (probBLOCKS-8-0), παρατηρείται ότι, παρά την αποδοτικότητα που του προσφέρει μια ισχυρή ευρετική συνάρτηση, ο **Best-First δεν εγγυάται βέλτιστη λύση**. Στην πράξη, ο Best-First βρήκε λύση με κόστος 10, ενώ ο BFS επιβεβαιώνει ότι υπάρχει καλύτερη λύση με κόστος 9.

Αυτό συμβαίνει επειδή η απόδοση του Best-First εξαρτάται αποκλειστικά από την ευρετική, η οποία αποτελεί μόνο εκτίμηση της απόστασης από τον στόχο, χωρίς να λαμβάνει υπόψη το πραγματικό κόστος διαδρομής όπως κάνει ο A*. Με άλλα λόγια, ο Best-First καθοδηγείται μόνο από την “εκτίμηση” και όχι από ένα συνδυασμό εκτίμησης και πραγματικού κόστους.

Αρχική κατάσταση τύπου 1 (ενιαία στοίβα)

Ο αλγόριθμος και σε αυτήν την περίπτωση σταματά να βρίσκει λύσεις για προβλήματα από το probBLOCKS-35-1 και μετά, δηλαδή για περιπτώσεις με 35 αντικείμενα και άνω. Παρακάτω παρατίθενται ενδεικτικά παραδείγματα εκτέλεσης:

```
C:\mathimata\BlocksWorld>python bw.py best probBLOCKS-20-1.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 37

Κόμβοι που επεκτάθηκαν: 182

Μέγιστο βάθος αναζήτησης: 37

Χρόνος εκτέλεσης: 0.5067408084869385

Η λύση είναι έγκυρη: NAI

```
C:\mathimata\BlocksWorld>python bw.py best probBLOCKS-25-1.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 46

Κόμβοι που επεκτάθηκαν: 310

Μέγιστο βάθος αναζήτησης: 46

Χρόνος εκτέλεσης: 4.776403188705444

Η λύση είναι έγκυρη: NAI

```
C:\mathimata\BlocksWorld>python bw.py best probBLOCKS-28-1.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 52

Κόμβοι που επεκτάθηκαν: 399

Μέγιστο βάθος αναζήτησης: 52

Χρόνος εκτέλεσης: 3.6129705905914307

Η λύση είναι έγκυρη: NAI

```
C:\mathimata\BlocksWorld>python bw.py best probBLOCKS-30-1.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 56

Κόμβοι που επεκτάθηκαν: 458

Μέγιστο βάθος αναζήτησης: 56

Χρόνος εκτέλεσης: 5.108535289764404

Η λύση είναι έγκυρη: NAI

Σε αυτόν τον τύπο προβλημάτων, η ευρετική συνάρτηση h_2 φαίνεται να αξιοποιεί πλήρως τις δυνατότητές της. Αυτό φαίνεται και από τα αποτελέσματα, καθώς ο αλγόριθμος δεν αντιμετώπισε μεγαλύτερη δυσκολία σε σχέση με τα προηγούμενα προβλήματα, όπου όλα τα αντικείμενα ξεκινούσαν πάνω στο τραπέζι. Μάλιστα, σε αρκετές περιπτώσεις οι χρόνοι εκτέλεσης ήταν ακόμη καλύτεροι.

Επομένως, συμπεραίνεται πως μια καλή ευρετική συνάρτηση μπορεί να γλυτώσει τον Best από το να ακολουθήσει τα βήματα του DFS, και να του επιτρέψει να αδιαφορεί για τον τύπο της αρχικής κατάστασης ενός προβλήματος Blocks World.

Αρχική κατάσταση τύπου 2 (μερικές στοίβες)

Σε αυτήν την περίπτωση, ο Best-First Search κατάφερε να λύσει όλα τα δοσμένα προβλήματα, κυρίως επειδή αυτά περιλαμβάνουν μέχρι 11 αντικείμενα. Δεδομένου ότι

τα προβλήματα είναι σχετικά μικρά, δεν προκύπτουν πολλά νέα συμπεράσματα. Όσα έχουν εξαχθεί έως τώρα παραμένουν έγκυρα και σε αυτή την κατηγορία.

```
C:\mathimata\BlocksWorld>python bw.py best probBLOCKS-11-2.pddl sol.txt  
SUCCESS  
Κόστος διαδρομής: 19  
Κόμβοι που επεκτάθηκαν: 60  
Μέγιστο βάθος αναζήτησης: 19  
Χρόνος εκτέλεσης: 0.0370488166809082  
Η λύση είναι έγκυρη: ΝΑΙ
```

Ο ΑΛΓΟΡΙΘΜΟΣ ΑΝΑΖΗΤΗΣΗΣ ΠΡΩΤΑ ΣΤΟΝ ΚΑΛΥΤΕΡΟ ΜΠΟΡΕΙ ΝΑ ΕΠΙΛΥΣΕΙ ΟΛΑ ΤΑ ΠΡΟΒΛΗΜΑΤΑ ΜΕ ΕΩΣ 35 ΚΑΙ ΚΑΤΩ ΑΝΤΙΚΕΙΜΕΝΑ. ΜΕ ΤΗΝ ΣΥΓΚΕΚΡΙΜΕΝΗ ΕΥΡΕΤΙΚΗ ΣΥΝΑΡΤΗΣΗ, ΑΔΙΑΦΟΡΕΙ ΓΙΑ ΤΟΝ ΤΥΠΟ ΠΡΟΒΛΗΜΑΤΟΣ, ΚΑΙ ΠΑΡΟΛΟ ΠΟΥ ΔΕΝ ΕΓΓΥΑΤΑΙ ΒΕΛΤΙΣΤΗ ΛΥΣΗ, ΕΙΝΑΙ ΕΞΑΙΡΕΤΙΚΑ ΑΠΟΔΟΤΙΚΟΣ

#==== ΑΝΑΖΗΤΗΣΗ A* ====#

Ο A* μπορεί να επιλύσει αρκετά μικρότερο εύρος προβλημάτων σε σχέση με τον Best. Για την επίδειξη της λειτουργίας του θα χρησιμοποιηθεί ένα δείγμα από τα προβλήματα με ουσιαστικό (όχι μικρό) πλήθος αντικειμένων.

Αρχική κατάσταση τύπου 0 (όλα τα αντικείμενα στο τραπέζι)

Ο A* στον τύπο προβλημάτων 0, μπορεί να επιλύσει μέχρι το πρόβλημα probBLOCKS-14-0, χωρίς όμως να επιλύει το probBLOCKS-13-0. Ήταν προβλεπόμενη μια μείωση της αποδοτικότητας σε σχέση με τον Best, αλλά το συγκεκριμένο χάσμα ξεπέρασε τις αρχικές χοντρικές υποθέσεις. Ακολουθούν παραδείγματα

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-9-0.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 19

Κόμβοι που επεκτάθηκαν: 13360

Μέγιστο βάθος αναζήτησης: 19

Χρόνος εκτέλεσης: 1.9185705184936523

Η λύση είναι έγκυρη: NAI

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-10-0.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 20

Κόμβοι που επεκτάθηκαν: 11376

Μέγιστο βάθος αναζήτησης: 20

Χρόνος εκτέλεσης: 1.9879019260406494

Η λύση είναι έγκυρη: NAI

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-11-0.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 19

Κόμβοι που επεκτάθηκαν: 7574

Μέγιστο βάθος αναζήτησης: 19

Χρόνος εκτέλεσης: 1.245887041091919

Η λύση είναι έγκυρη: NAI

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-12-0.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 18

Κόμβοι που επεκτάθηκαν: 42433

Μέγιστο βάθος αναζήτησης: 20

Χρόνος εκτέλεσης: 10.017534494400024

Η λύση είναι έγκυρη: NAI

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-13-0.pddl sol.txt  
FAILURE
```

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-14-0.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 24

Κόμβοι που επεκτάθηκαν: 69410

Μέγιστο βάθος αναζήτησης: 24

Χρόνος εκτέλεσης: 17.475966453552246

Η λύση είναι έγκυρη: NAI

Σε αντίθεση με τον Best, η συνολική εκτίμηση για τον A* δίνεται από τη συνάρτηση $f(n)=g(n)+h(n)$ όπου:

- **$g(n)$** = κόστος από την αρχική κατάσταση μέχρι το n ,
- **$h(n)$** = ευρετική εκτίμηση από το n μέχρι τον στόχο ($h3$ στην υλοποίηση)

Όπως αναφέρθηκε προηγουμένως, η συνάρτηση εκτίμησης του A* (εφόσον πληροί τα απαραίτητα κριτήρια) καθιστά τον αλγόριθμο βέλτιστο, καθώς επεκτείνει κατά προτεραιότητα τις καταστάσεις που θεωρούνται πιο υποσχόμενες, λαμβάνοντας υπόψη

τόσο την πρόοδο που έχει ήδη επιτευχθεί όσο και την εκτιμώμενη απόσταση από τον στόχο. Με άλλα λόγια, ο A* επιτυγχάνει τον συνδυασμό πραγματικού κόστους και εκτίμησης μέσω της σχέσης $f(n) = g(n) + h(n)$.

Ωστόσο, η βελτιστοτητα αυτή έχει κόστος, καθώς απαιτεί σημαντικά περισσότερους πόρους σε χρόνο και μνήμη σε σύγκριση με τον Best-First Search. Αυτός είναι και ο λόγος που παρατηρήθηκε σημαντική μείωση του A* σε σχέση με τον Best ως προς το εύρος των προβλημάτων που μπορεί να λύσει ο καθένας, αλλά και σε γενικότερης αποδοτικότητας. Ο A* μπορεί να επεκτείνει πολύ περισσότερους κόμβους από best όταν το frontier/heap γίνεται μεγάλο. Αυτό προκαλεί timeouts (FAILURE) σε κάποια παραδείγματα.

Ο σχεδιαστής της ευρετικής μπορεί να προσαρμόσει το βάρος των συνιστωσών $g(n)$ και $h(n)$ ώστε να επιτύχει την επιθυμητή ισορροπία ανάμεσα σε βελτιστοτητα και αποδοτικότητα, ανάλογα με τις ανάγκες του εκάστοτε προβλήματος.

Αρχική κατάσταση τύπου 1 (ενιαία στοίβα)

Ο A* στον τύπο προβλημάτων 1, μπορεί να επιλύσει μέχρι το πρόβλημα probBLOCKS-14-1, χωρίς όμως να επιλύει το probBLOCKS-13-1

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-9-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 14
Κόμβοι που επεκτάθηκαν: 534
Μέγιστο βάθος αναζήτησης: 14
Χρόνος εκτέλεσης: 0.04596734046936035
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld> python bw.py astar probBLOCKS-10-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 17
Κόμβοι που επεκτάθηκαν: 7999
Μέγιστο βάθος αναζήτησης: 18
Χρόνος εκτέλεσης: 1.1891982555389404
Η λύση είναι έγκυρη: NAI
```

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-11-1.pddl sol.txt
SUCCESS
Κόστος διαδρομής: 18
Κόμβοι που επεκτάθηκαν: 4876
Μέγιστο βάθος αναζήτησης: 18
Χρόνος εκτέλεσης: 0.6977171897888184
Η λύση είναι έγκυρη: NAI
```



```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-12-1.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 22

Κόμβοι που επεκτάθηκαν: 4370

Μέγιστο βάθος αναζήτησης: 22

Χρόνος εκτέλεσης: 0.7289292812347412

Η λύση είναι έγκυρη: NAI

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-13-1.pddl sol.txt  
FAILURE
```

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-14-1.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 25

Κόμβοι που επεκτάθηκαν: 128646

Μέγιστο βάθος αναζήτησης: 25

Χρόνος εκτέλεσης: 37.10933256149292

Η λύση είναι έγκυρη: NAI

Αρχική κατάσταση τύπου 2 (μερικές στοίβες)

Σε αυτήν την περίπτωση, ο A* κατάφερε να λύσει όλα τα δοσμένα προβλήματα, κυρίως επειδή αυτά περιλαμβάνουν μέχρι 11 αντικείμενα. Όσα συμπεράσματα έχουν εξαχθεί έως τώρα παραμένουν έγκυρα και σε αυτή την κατηγορία.

```
C:\mathimata\BlocksWorld>python bw.py astar probBLOCKS-11-2.pddl sol.txt  
SUCCESS
```

Κόστος διαδρομής: 20

Κόμβοι που επεκτάθηκαν: 15951

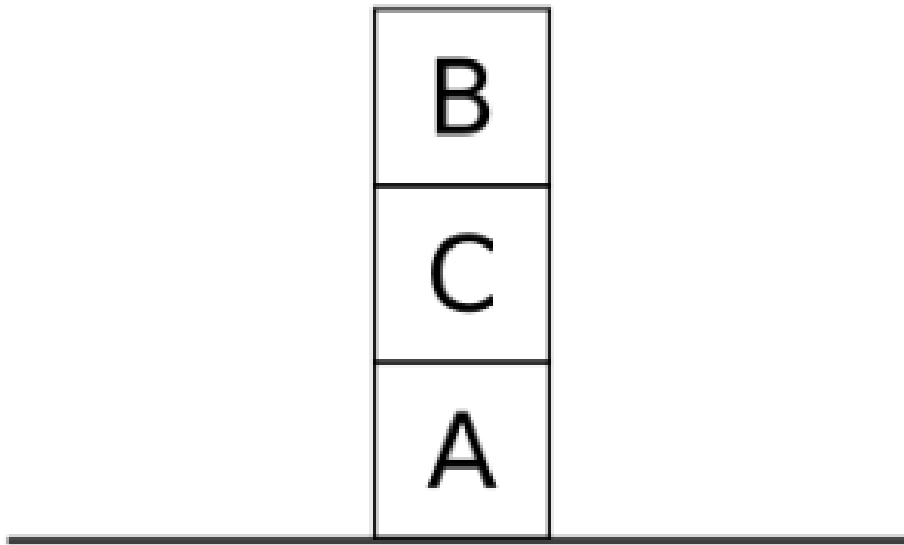
Μέγιστο βάθος αναζήτησης: 20

Χρόνος εκτέλεσης: 3.2442729473114014

Η λύση είναι έγκυρη: NAI

Ο ΑΛΓΟΡΙΘΜΟΣ A* ΕΙΝΑΙ ΒΕΛΤΙΣΤΟΣ ΟΤΑΝ Η ΕΥΡΕΤΙΚΗ ΣΥΝΑΡΤΗΣΗ ΕΙΝΑΙ ΠΑΡΑΔΕΚΤΗ. ΩΣΤΟΣΟ, ΠΛΗΡΩΝΕΙ ΑΥΤΗ ΤΗΝ ΙΚΑΝΟΤΗΤΑ ΜΕ ΥΨΗΛΗ ΧΩΡΙΚΗ ΚΑΙ ΧΡΟΝΙΚΗ ΠΟΛΥΠΛΟΚΟΤΗΤΑ, ΜΕ ΑΠΟΤΕΛΕΣΜΑ ΝΑ ΜΗΝ ΜΠΟΡΕΙ ΝΑ ΛΥΣΕΙ ΜΕΓΑΛΑ ΠΡΟΒΛΗΜΑΤΑ (ΞΕΠΕΡΝΩΝΤΑΣ ΠΕΡΙΠΟΥ ΤΑ 12-14 ΑΝΤΙΚΕΙΜΕΝΑ ΣΤΙΣ ΔΟΚΙΜΕΣ).#

#==== ΣΥΜΠΕΡΑΣΜΑΤΑ ====#



Αρχικά, οι **αλγόριθμοι τυφλής αναζήτησης**, όπως ο BFS και ο DFS, παρουσιάζουν διαφορετικά χαρακτηριστικά που καθορίζουν την επιλογή τους ανάλογα με το πρόβλημα. Ο BFS είναι βέλτιστος και εγγυάται την εύρεση της λύσης με το μικρότερο δυνατό κόστος, αλλά πληρώνει το τίμημα της μεγαλύτερης χρήσης μνήμης και χρόνου, ενώ ο DFS είναι πολύ αποδοτικός ως προς τη μνήμη, χωρίς όμως να εγγυάται βέλτιστο αποτέλεσμα. Η επιλογή μεταξύ τους εξαρτάται ουσιαστικά από το αν προτεραιότητα έχει η **βελτιστότητα** ή η **απόδοση** (χώρος) στο εκάστοτε πρόβλημα.

Στη συνέχεια, οι **αλγόριθμοι πληροφορημένης αναζήτησης**, όπως ο A* και ο Best, αξιοποιούν ευρετικές συναρτήσεις για να καθοδηγήσουν την αναζήτηση. Ο A* συνδυάζει το πραγματικό κόστος και την εκτίμηση της απόστασης από τον στόχο, εξασφαλίζοντας βέλτιστη λύση όταν η ευρετική είναι παραδεκτή, ενώ ο Best αναζητά τις πιο υποσχόμενες καταστάσεις μόνο βάσει της ευρετικής, προσφέροντας υψηλή αποδοτικότητα αλλά χωρίς εγγύηση βέλτιστης λύσης. Για αυτόν ακριβώς τον λόγο, ο Best αποδείχθηκε ο πιο αποδοτικός ανάμεσα στους τέσσερις αλγορίθμους.

Η **ποιότητα της ευρετικής** είναι καθοριστική και για τους δύο αλγορίθμους. Μια καλά σχεδιασμένη ευρετική μπορεί να μειώσει δραστικά τον αριθμό των κόμβων που εξετάζονται, αυξάνοντας την ταχύτητα και την αποδοτικότητα, ενώ κακές ή ανεπαρκείς ευρετικές οδηγούν σε σημαντική επιβάρυνση.

Αναφορικά με τους **τύπους προβλημάτων**, οι μη βελτιστοί αλγόριθμοι είναι πιο ευαίσθητοι στη γενική δυσκολία και στη δομή του προβλήματος, ανεξαρτήτως του αριθμού των αντικειμένων, και μπορεί να καθυστερήσουν ή να αποτύχουν σε σύνθετες διατάξεις. Αντίθετα, οι αλγόριθμοι πληροφορημένης αναζήτησης, και ιδιαίτερα ο Best,

μπορούν να παρακάμψουν μεγάλο μέρος της αναζήτησης χάρη σε μια ισχυρή ευρετική, αντιμετωπίζοντας αποτελεσματικά ακόμη και προβλήματα αυξημένης δυσκολίας.

Τέλος, κάθε αλγόριθμος παρουσιάζει διαφορετικές προκλήσεις ως προς την **υλοποίηση**, την **αποδοτικότητα (πολυπλοκότητα)**, και τη **βελτιστότητα**. Η επιλογή του κατάλληλου αλγορίθμου εξαρτάται από τη φύση του προβλήματος, τους στόχους (ταχύτητα ή βέλτιστη λύση) και τους περιορισμούς μνήμης και χρόνου που ισχύουν.